রিয়েল-টাইম মেসেজিং মডিউল

প্রযুক্তিগত নথি ও আর্কিটেকচার গাইড

Laravel + WebSocket + Bootstrap 5

সংস্করণ	v2.0 - গার্ড-ভিত্তিক পলিমরফিক
তারিখ	০৬ এপ্রিল, ২০২৬
ব্যাপ্তি	অ্যাডমিন ↔ ইউজার একে-এক মেসেজিং
স্ট্যাক	Laravel 10/11, beyondcode/laravel-websockets, Bootstrap 5
মডিউল	nwidart/laravel-মডিউলs - পুনঃব্যবহারযোগ্য
অবস্থা	প্রোডাকশন রেডি

এই নথি রিয়েল-টাইম মেসেজিং মডিউলের প্রযুক্তিগত কাঠামো, নকশাগত সিদ্ধান্ত, ফাইল গঠন, ডেটাবেস, নিরাপত্তা এবং ডিপ্লয়মেন্ট পরিষ্কারভাবে তুলে ধরে। এটি এমনভাবে সাজানো হয়েছে যাতে সিনিয়র বা টিম লিডের সামনে প্রজেক্টের উদ্দেশ্য, নির্মাণ-পদ্ধতি এবং ব্যবসায়িক সুবিধা সহজে ব্যাখ্যা করা যায়।

এই ডকুমেন্টটি গোপনীয় এবং শুধুমাত্র অভ্যন্তরীণ রিভিউর জন্য

১. পূর্ণ প্রজেক্ট ওভারভিউ

এই মডিউলটি একটি একে-এক রিয়েল-টাইম মেসেজিং সিস্টেম, যেখানে অ্যাডমিন এবং ইউজার সরাসরি চ্যাট করতে পারে। উদ্দেশ্য হলো সাপোর্ট, কমিউনিকেশন এবং ইন-অ্যাপ নোটিফিকেশনকে দ্রুত, নির্ভরযোগ্য এবং পুনঃব্যবহারযোগ্যভাবে বাস্তবায়ন করা।

প্রজেক্টের মূল ধারণা

কেন বানানো হচ্ছে	সাপোর্ট টিম যেন একই প্ল্যাটফর্মে তাৎক্ষণিক উত্তর দিতে পারে।
কী সমস্যা সমাধান করে	পেজ রিলোড ছাড়া মেসেজ, অপঠিত অবস্থা, পড়া হয়েছে সূচক এবং নতুন বার্তা সতর্কতা দেখায়।
কার জন্য	অ্যাডমিন, ইউজার, এবং ভবিষ্যতে restaurant বা deliveryman-এর মতো আলাদা গার্ড-ভিত্তিক রোল।
কীভাবে গড়া হয়েছে	Laravel, WebSocket ব্রডকাস্ট, প্রাইভেট চ্যানেল authorization, গার্ড শনাক্তকারী helper, এবং মডুলার structure ব্যবহার করে।
কী সুবিধা দেয়	দ্রুত response, লাইভ সাপোর্ট, কেন্দ্রীভূত কথোপকথন, এবং ভবিষ্যৎ-প্রস্তুত reuse।

সংক্ষিপ্ত সারাংশ

- অ্যাডমিন ↔ ইউজার একে-এক চ্যাট
- রিয়েল-টাইম মেসেজ পাঠানো ও গ্রহণ
- ইন-অ্যাপ সতর্কতা
- পড়া হয়েছে সূচক ও unread count

এই ডকুমেন্টে এমনভাবে বিষয়গুলো সাজানো হয়েছে যেন সিনিয়র বা টিম লিডের সামনে প্রজেক্টের উদ্দেশ্য, নির্মাণ-পদ্ধতি এবং ভবিষ্যৎ সম্প্রসারণ সহজে ব্যাখ্যা করা যায়।

২. প্রযুক্তি স্ট্যাক

ব্যবহৃত প্রযুক্তি

ব্যাকএন্ড ফ্রেমওয়ার্ক	Laravel 10 / 11 - মূল অ্যাপ্লিকেশন ফ্রেমওয়ার্ক
WebSocket সার্ভার	beyondcode/laravel-websockets - self-hosted এবং Pusher-compatible
Module System	nwidart/laravel-মডিউলস - reusable মডুলার architecture
রিয়েল-টাইম ক্লায়েন্ট	Laravel Echo - ইভেন্ট listener
WebSocket JS	Pusher JS - browser-side connection
ফ্রন্টএন্ড	Bootstrap 5 - UI framework
Icons	Bootstrap Icons - icon library
Queue Driver	Database / Redis - async ব্রডকাস্ট processing
Auth	Laravel guard-based প্রমাণীকরণ - web, admin, restaurant, deliveryman

কেন **beyondcode/laravel-websockets**

নির্বাচনের কারণ

খরচ	Self-hosted হওয়ায় আলাদা cloud fee নেই
ডেটা নিয়ন্ত্রণ	সার্ভারের ভেতরেই connection handle করা যায়
সেটআপ	Laravel প্রজেক্টের সাথে সরাসরি configure করা যায়
কম্প্যাটিবিলিটি	Pusher-compatible হওয়ায় Echo stack সহজে কাজ করে
প্রোডাকশন	Supervisor-এর মাধ্যমে production-readyভাবে চালানো যায়

৩. সিস্টেম আর্কিটেকচার

চ্যাট পাঠানোর সময় HTTP request প্রথমে message ডাটাবেজে save হয়, তারপর event ব্রডকাস্ট হয়, WebSocket server প্রাইভেট চ্যানেল-এ সেই event push করে, এবং recipient browser তা Laravel Echo দিয়ে লাইভভাবে পায়। ফলে কোনো পৃষ্ঠা রিলোড ছাড়াই UI update হয়।

রিয়েল-টাইম ফ্লো

ধাপ ১	ইউজার বা অ্যাডমিন POST request পাঠায় /messaging/{id}/messages এ
ধাপ ২	MessageController message সংরক্ষণ করে
ধাপ ৩	MessageSent event fire হয়
ধাপ ৪	WebSocket server conversation.{id} প্রাইভেট চ্যানেল-এ push করে
ধাপ ৫	Recipient browser Laravel Echo listener দিয়ে receive করে
ধাপ ৬	UI সাথে সাথে append হয়, পৃষ্ঠা রিলোড লাগে না
ধাপ ৭	NewMessageNotification toast দেখায় page-এর উপর

গার্ড-ভিত্তিক পলিমরফিক সিদ্ধান্ত

একই আইডি বিভিন্ন টেবিলে থাকতে পারে - যেমন users.id = 1 এবং admins.id = 1। তাই শুধু auth()->id() ব্যবহার করলে বিভ্রান্তি তৈরি হয়। এই কারণেই participant_type এবং participant_id একসাথে ব্যবহার করা হয়েছে। এতে guard আলাদা করা যায়, এবং ভবিষ্যতে restaurant বা deliveryman যোগ করলেও কাঠামো নষ্ট হয় না।

- participant_id + participant_type ব্যবহার করে identity আলাদা করা হয়েছে।
- AuthParticipant helper active guard detect করে।
- নতুন role যোগ করতে হলে config update করলেই চলে।

৪. ডেটাবেস ডিজাইন

ডেটাবেসকে এমনভাবে সাজানো হয়েছে যাতে conversation, participants, এবং message - এই তিনটি স্তর পরিষ্কার থাকে। এতে unread count, access control, এবং future file support সহজ হয়।

৪.১ conversations

id	স্বতন্ত্র conversation ID
created_at	conversation তৈরির সময়
updated_at	শেষ আপডেটের সময়

৪.২ conversation_participants

conversation_id	কোন conversation-এ অংশ নিচ্ছে
participant_id	অংশগ্রহণকারীর ID
participant_type	user / admin / future role
last_read_at	শেষ কোন message পর্যন্ত পড়েছে
joined_at	কখন conversation-এ যুক্ত হয়েছে
unique constraint	conversation_id + participant_id + participant_type

৪.৩ messages

conversation_id	message কোন conversation-এর
sender_id / sender_type	কে পাঠিয়েছে
body	মূল message content
type	text, file, image
file_path / file_name	future file sharing support
is_deleted	soft delete flag
read_at	পড়া হয়েছে কি না বোঝায়
created_at	পাঠানোর সময়

৫. মডিউল ফাইল স্ট্রাকচার

এই প্রজেক্টকে Modules/Messaging ফোল্ডারের ভিতরে আলাদা মডিউল হিসেবে রাখা হয়েছে, যাতে copy করে অন্য Laravel প্রজেক্টেও ব্যবহার করা যায়।

মূল ডিরেক্টরি ও দায়িত্ব

Config	config.php - user model, guard, field mapping
Database/Migrations	conversations, participants, messages টেবিল
Events	MessageSent, MessageRead, NewMessageNotification, UserTyping
Helpers	AuthParticipant.php - active guard detect
Http/Controllers	ConversationController.php, MessageController.php
Models	Conversation, ConversationParticipant, Message
Providers	MessagingServiceProvider.php
Resources/views	admin inbox, user chat window, shared partials
Routes	web.php এবং channels.php

৬. মূল কম্পোনেন্ট ব্যাখ্যা

AuthParticipant Helper

এই helper active guard চিনে current user-এর type, id, model, name, এবং প্রমাণীকরণ state রিটার্ন করে। এর ফলে code base-এ বারবার guard logic লিখতে হয় না।

type()	বর্তমান guard-এর ধরন রিটার্ন করে
id()	বর্তমান লগইনকৃত user-এর ID
model()	আসল Eloquent model object
name()	display name
check()	authenticated কিনা তা যাচাই করে

Events ও Broadcasting

MessageSent	মেসেজ পাঠালে ব্রডকাস্ট হয়
MessageRead	মেসেজ পড়া হলে double tick update করে
NewMessageNotification	toast নোটিফিকেশন দেখায়
UserTyping	future typing indicator-এর জন্য প্রস্তুত

Private Channel Authorization

channel authorization-এ শুধু সেই participant access পাবে যে conversation_participants টেবিলে যুক্ত আছে। এতে unauthorized user conversation data দেখতে পারবে না।

৭. ব্যবহারকারী প্রবাহ

ইউজার সাইড

- ইউজার Message Support এ ক্লিক করে chat page খুলে।
- Controller auto-create বা existing conversation খুঁজে বের করে।
- Chat window খুললেই last_read_at update হয়।
- ইউজার মেসেজ পাঠালে তা সাথে সাথে অ্যাডমিন স্ক্রিনে পৌঁছে যায়।
- অ্যাডমিন inactive tab-এ থাকলেও toast নোটিফিকেশন পায়।

অ্যাডমিন সাইড

- অ্যাডমিন /messaging/ এ ঢুকলে user list ও unread count দেখতে পায়।
- conversation click করলে chat window খুলে।
- reply দিলে event ব্রডকাস্ট হয়।
- ইউজারের পাশে unread badge ও পড়া হয়েছে সূচক update হয়।

৮. ইন-অ্যাপ নোটিফিকেশন সিস্টেম

Web Push এখানে প্রধান লক্ষ্য নয়। এই মডিউলে WebSocket-এর মাধ্যমে in-app নোটিফিকেশন দেওয়া হয়েছে, যাতে ইউজার বা অ্যাডমিন খোলা পেজে থাকলেই তাৎক্ষণিক alert দেখতে পায়।

Notification behavior

চ্যাট উইন্ডো খোলা থাকলে	নতুন message সরাসরি chat area-তে append হয়
অন্য **page** খোলা থাকলে	toast নোটিফিকেশন দেখায়
admin inbox-এ	unread badge এবং list update হয়
browser বন্ধ থাকলে	এই scope-এর বাইরে রাখা হয়েছে

গ্লোবাল **listener**

chat-layout.blade.php এ global Echo listener রাখা হয়েছে, যাতে সক্রিয় পেজে chat না থাকলেও নতুন message detect করা যায়। active conversation হলে duplicate নোটিফিকেশন এড়ানো হয়।

৯. পুনঃব্যবহারযোগ্যতা

এই মডিউল copy করে অন্য Laravel প্রজেক্টে নেয়া যায়। config/messaging.php এ নতুন participant type যোগ করলেই codebase অনেকটাই একই থাকে।

অন্য প্রজেক্টে বসানোর ধাপ

ধাপ ১	Modules/Messaging/ ফোল্ডার কপি করা
ধাপ ২	composer autoload এ Modules namespace যোগ করা
ধাপ ৩	ServiceProvider register করা
ধাপ ৪	config/messaging.php update করা
ধাপ ৫	migrate এবং websockets সার্ভার চালানো

১০. ডিপ্লয়মেন্ট

প্রয়োজনীয় সার্ভিস

WebSocket Server	php artisan websockets:serve - Supervisor এ চালানো
Queue Worker	php artisan queue:work - background ব্রডকাস্ট
Web Server	Nginx + PHP-FPM
Production	wss:// ব্যবহার করা উচিত

১১. নিরাপত্তা চেকলিস্ট

- Private channel authorization বাধ্যতামূলক।
- Conversation access middleware দিয়ে 403 protect করা হয়েছে।
- Message sender ownership যাচাই করা হয়েছে।
- Input validation আছে; body length সীমা নির্ধারণ করা হয়েছে।
- Frontend এ HTML escape করে XSS risk কমানো হয়েছে।
- AJAX request-এ CSRF token ব্যবহার করা হয়েছে।
- Production-এ SSL/TLS সহ wss:// দরকার।

১২. ভবিষ্যৎ রোডম্যাপ

ভবিষ্যৎ ফিচার

Typing Indicator	প্রায় প্রস্তুত; frontend keydown trigger বাকি
File Sharing	schema প্রস্তুত; upload UI ও validation বাকি
Restaurant / Deliveryman Chat	architecture ready; শুধু config entry যোগ করতে হবে
Message Search	এখনো শুরু হয়নি
Browser Push Notification	future phase
Message Reactions	future feature

১৩. দ্রুত রেফারেন্স - রুট ও কমান্ড

Routes

GET /messaging/	ইউজারের জন্য auto open; অ্যাডমিনের জন্য inbox list
GET /messaging/{id}	chat window open
POST /messaging/{id}/messages	message send
POST /messaging/{id}/read	mark as read
GET /messaging/{id}/messages/more	older messages load

Commands

ইনস্টল	composer require beyondcode/laravel-websockets nwidart/laravel-মডিউলs
ফ্রন্টএন্ড	npm install laravel-echo pusher-js
ডেভেলপমেন্ট	php artisan websockets:serve
কিউ	php artisan queue:work
প্রোডাকশন	Supervisorctl start laravel-websockets / laravel-queue

শেষ কথা - এই মডিউলটি production-level, reuse-friendly, এবং ভবিষ্যৎ-নিরাপদভাবে তৈরি করা হয়েছে। সোজা বাংলায়: এখন আলাদা আলাদা প্রজেক্টে chat logic আবার নতুন করে লিখতে হবে না।